

Termin und Erfüllungsort

Der Termin legt fest, bis wann eine Leistung oder Aufgabe erbracht werden muss. Der Erfüllungsort bestimmt den Ort, an dem die Leistung vertragsgemäß zu erbringen ist. Beide Aspekte sind in Verträgen oder Projektplänen geregelt und beeinflussen die rechtzeitige Lieferung sowie mögliche Haftungsfragen.

7.8 Bedarfsanalyse

Eine Bedarfsanalyse ermittelt den tatsächlichen Bedarf und Anforderungen als Basis für Entscheidungen und Projektplanung Vorgehen:

- **Erhebung:** Daten sammeln (z.B. Interviews, Umfragen)
- **Analyse:** Anforderungen bewerten und priorisieren
- **Dokumentation:** Ergebnisse festhalten und mit relevanten Stakeholdern abstimmen

7.9 Lasten- und Pflichtenheft

Lastenheft

Ein Lastenheft ist ein Dokument des Auftraggebers, welches dieser alleine, oder in Zusammenarbeit mit dem Auftragnehmer erstellt. Es stellt die Anforderungen und Wünsche des Kunden dar und enthält technische und nicht-technische Bestandteile aus der Sicht des Kunden. Das Lastenheft ist zudem oft die Grundlage eines Werkvertrags.

Pflichtenheft

Im Gegensatz zum Lastenheft ist das Pflichtenheft ein Dokument, welches der Auftragnehmer aufgrund des Lastenhefts erstellt. Es stellt die konkrete technische Lösung dar mit detaillierten Spezifikationen.

7.10 Kundenvorgaben bei der Leistungserbringung

Kundenvorgaben sind spezifische Anforderungen, die ein Kunde an ein Produkt oder eine Dienstleistung stellt. Sie beeinflussen maßgeblich die Projektplanung, -durchführung und -kontrolle.

Arten von Kundenvorgaben:

- **Funktionale Anforderungen:** Beschreiben die gewünschten Funktionen und Eigenschaften des Produkts.
- **Nicht-funktionale Anforderungen:** Definieren Qualitätsmerkmale wie Leistung, Sicherheit oder Skalierbarkeit.
- **Technische Vorgaben:** Spezifikationen zu Hardware, Software oder Schnittstellen.
- **Rechtliche & regulatorische Vorgaben:** Compliance-Anforderungen, Datenschutz (z. B. DSGVO), Zertifizierungen.
- **Budget- und Zeitvorgaben:** Festgelegte Kostenrahmen und Liefertermine.

Bedeutung für das Projektmanagement:

- **Anforderungsanalyse:** Klärung und Dokumentation der Vorgaben zu Beginn des Projekts.
- **Pflichtenheft:** Übersetzung der Kundenanforderungen in technische und organisatorische Spezifikationen.
- **Risikomanagement:** Identifikation und Bewertung von Risiken im Zusammenhang mit den Vorgaben.
- **Qualitätsmanagement:** Sicherstellung, dass die Anforderungen erfüllt werden (z. B. durch Tests, Reviews).
- **Change Management:** Umgang mit nachträglichen Änderungen der Kundenvorgaben.

Herausforderungen & Lösungsansätze:

Herausforderung	Lösungsansatz
Unklare oder unvollständige Anforderungen	Frühzeitige Klärung, Workshops mit dem Kunden
Widersprüchliche Vorgaben	Abstimmung und Priorisierung mit dem Kunden
Änderungen während des Projekts	Flexibles Change Management, dokumentierte Anpassungen
Enge Zeit- und Budgetvorgaben	Realistische Planung, regelmäßige Fortschrittskontrollen

7.11 Technische Voraussetzungen

Technische Voraussetzungen sind die notwendigen Hard- und Softwareanforderungen, die erfüllt sein müssen, damit ein System, eine Anwendung oder ein Dienst ordnungsgemäß funktioniert.

Wichtige Aspekte technischer Voraussetzungen

Betriebssystem

Das Betriebssystem muss mit der Software kompatibel sein.

Wichtige Faktoren:

- **Typ:** Windows, Linux, macOS, Unix, Android, iOS
- **Version:** Mindestanforderungen an die OS-Version
- **Architektur:** 32-Bit oder 64-Bit
- **Treiber & Updates:** Notwendige Aktualisierungen oder spezielle Treiber

Hersteller

Spezifische Anforderungen an Hardware oder Software von bestimmten Herstellern:

- **Kompatibilität:** Bestimmte Software läuft nur auf zertifizierter Hardware (z. B. Apple-Produkte)
- **Support & Garantie:** Herstellerabhängige Serviceleistungen
- **Lizenzmodelle:** Unterschiedliche Lizenzbedingungen je nach Hersteller

Hardware-Anforderungen

Mindest- und empfohlene Spezifikationen für die Nutzung einer Software oder eines Systems:

- **Prozessor (CPU):** Mindesttaktfrequenz, Anzahl der Kerne
- **Arbeitsspeicher (RAM):** Mindestgröße für reibungslosen Betrieb
- **Grafikkarte (GPU):** Anforderungen für grafikintensive Anwendungen
- **Festplattenspeicher:** Mindestfreier Speicherplatz für Installation und Betrieb

Netzwerkanforderungen

Voraussetzungen für die Verbindung mit Netzwerken oder Cloud-Diensten:

- **Bandbreite:** Mindestgeschwindigkeit für Datenübertragungen
- **Latenz:** Anforderungen für Echtzeitanwendungen (z. B. VoIP, Gaming)
- **Sicherheitsrichtlinien:** Firewall-Regeln, VPN, Proxy-Unterstützung

Softwareabhängigkeiten

Bestimmte Anwendungen benötigen zusätzliche Software-Komponenten:

- **Frameworks & Bibliotheken:** .NET, Java, Python, DirectX
- **Datenbanken:** MySQL, PostgreSQL, MS SQL Server
- **Schnittstellen (APIs):** Kompatibilität mit externen Systemen

Sicherheitsanforderungen

Technische Maßnahmen zur Sicherstellung von Datenschutz und Systemsicherheit:

- **Verschlüsselung:** SSL/TLS, AES
- **Authentifizierung:** 2-Faktor-Authentifizierung, Single Sign-On (SSO)
- **Zugriffsrechte:** Rollenbasierte Berechtigungen

7.12 Einhaltung des Budgets

Die Einhaltung des Budgets bedeutet, dass die geplanten finanziellen Mittel eines Projekts nicht überschritten werden und die Kosten innerhalb des vorgegebenen Rahmens bleiben.

Maßnahmen zur Budgetkontrolle:

- **Detaillierte Kostenplanung:** Frühzeitige Kalkulation aller Aufwände (Personal, Material, Software, Lizenzen).
- **Regelmäßige Budgetüberprüfung:** Vergleich von geplanten und tatsächlichen Kosten durch Controlling.

- **Risikomanagement:** Identifikation potenzieller Kostensteigerungen und Definition von Gegenmaßnahmen.
- **Kostensenkungsstrategien:** Optimierung von Ressourcen, Verhandlungen mit Lieferanten, Vermeidung unnötiger Ausgaben.
- **Change Management:** Bewertung von Änderungen auf Budgetauswirkungen und frühzeitige Anpassung der Planung.

7.13 Veränderungsprozesse

Veränderungsprozesse (Change Management) sind geplante und gesteuerte Anpassungen in Unternehmen oder Projekten, um neue Strukturen, Technologien oder Arbeitsweisen erfolgreich zu implementieren.

Motivierte Herangehensweise

Ein erfolgreicher Veränderungsprozess erfordert eine positive und motivierte Haltung:

- **Klare Zielsetzung:** Definition der gewünschten Veränderung und erwarteten Vorteile.
- **Transparenz:** Kommunikation der Notwendigkeit und der Vorteile der Veränderung.
- **Führungskräfte als Vorbilder:** Vorgesetzte sollten den Wandel aktiv unterstützen.
- **Partizipation:** Mitarbeiter frühzeitig einbinden, um Akzeptanz zu fördern.

Identifizierung und Darstellung der Veränderung

Damit eine Veränderung erfolgreich umgesetzt werden kann, müssen folgende Aspekte geklärt werden:

- **Analyse des Ist-Zustands:** Wo steht das Unternehmen aktuell?
- **Definition des Soll-Zustands:** Welche Ziele sollen erreicht werden?
- **Gap-Analyse:** Welche Lücken bestehen zwischen Ist- und Soll-Zustand?
- **Kommunikationsstrategie:** Wie wird der Wandel erklärt und präsentiert?

Einbeziehung der Mitarbeiter

Die aktive Beteiligung der Mitarbeiter ist entscheidend für den Erfolg eines Veränderungsprozesses:

- **Offene Kommunikation:** Sorgen und Fragen der Mitarbeiter ernst nehmen.
- **Feedbackmechanismen:** Regelmäßige Rückmeldungen ermöglichen Anpassungen.
- **Mitarbeitermotivation:** Positive Anreize schaffen, um Akzeptanz zu fördern.
- **Verantwortung übertragen:** Mitarbeiter in Entscheidungsprozesse einbinden.

Mitarbeiterqualifizierung

Neue Prozesse oder Technologien erfordern oft neue Fähigkeiten:

- **Schulungen und Workshops:** Weiterbildungsmöglichkeiten für betroffene Mitarbeiter.
- **Coaching und Mentoring:** Individuelle Unterstützung durch erfahrene Kollegen.
- **E-Learning und Selbststudium:** Flexible Lernformate für eigenständige Weiterbildung.
- **Fortlaufende Qualifizierung:** Langfristige Entwicklung der Mitarbeitenden im Blick behalten.

Promoter, Bremser, Skeptiker und Widerständler

Verschiedene Mitarbeitertypen beeinflussen den Veränderungsprozess:

Typ	Beschreibung	Umgangsstrategie
Promoter	Unterstützen aktiv den Wandel und treiben ihn voran.	Als Change-Agents einsetzen, um andere zu motivieren.
Bremser	Verzögern bewusst oder unbewusst den Prozess.	Ursachen analysieren, gezielt einbinden und Bedenken ausräumen.
Skeptiker	Zweifeln an der Veränderung, sind	Transparenz schaffen, rationale Argumente liefern.

Typ	Beschreibung	Umgangsstrategie
	aber offen für Argumente.	
Widerständler	Lehnen den Wandel aktiv ab und arbeiten dagegen.	Offenes Gespräch suchen, Ängste und Widerstände adressieren.

Ursachen von Widerständen

Widerstand gegen Veränderungen ist normal und kann verschiedene Ursachen haben:

- **Angst vor dem Unbekannten:** Unsicherheit über neue Prozesse oder Technologien.
- **Verlustängste:** Sorge um Arbeitsplatz, Status oder Einfluss.
- **Fehlende Kommunikation:** Unklare Informationen über den Veränderungsprozess.
- **Schlechte Erfahrungen:** Frühere gescheiterte Veränderungen.
- **Zusätzlicher Aufwand:** Angst vor Mehrbelastung ohne erkennbaren Nutzen.

7.14 Leistungsübergabe

Die Leistungsübergabe bezeichnet den formalen Abschluss eines Projekts oder einer Lieferung, bei dem die erbrachte Leistung an den Kunden oder Auftraggeber übergeben wird. Dabei wird geprüft, ob die vereinbarten Anforderungen erfüllt wurden.

Abnahmeprotokoll

Das Abnahmeprotokoll dokumentiert die Übergabe der Leistung und enthält:

- **Projekt- oder Liefergegenstand:** Detaillierte Beschreibung der erbrachten Leistung.
- **Prüfkriterien:** Festgelegte Standards und Anforderungen, die erfüllt sein müssen.
- **Mängelliste:** Falls vorhanden, erfasste Abweichungen und Fristen zur Behebung.
- **Unterschriften:** Bestätigung durch Auftraggeber und Auftragnehmer.

Wichtig

Das Abnahmeprotokoll ist ein rechtlich relevantes Dokument und dient als Nachweis für die Erfüllung der vertraglichen Pflichten.

Mängel und Mängelarten

Ein Mangel liegt vor, wenn die erbrachte Leistung nicht den vereinbarten Anforderungen entspricht. Mängelarten:

- **Sachmangel:** Die Leistung weist physische oder funktionale Defekte auf.
- **Rechtsmangel:** Dritte haben Ansprüche an der gelieferten Ware (z. B. Patentrechte).
- **Offene Mängel:** Sofort erkennbare Fehler bei der Abnahme.
- **Versteckte Mängel:** Erst später auftretende oder entdeckte Fehler.

Schlechtleistung

Schlechtleistung liegt vor, wenn eine Leistung zwar erbracht, aber nicht vertragsgemäß ausgeführt wurde. Beispiele:

- Fehlerhafte Verarbeitung (z. B. ungenaue Fertigung bei einem Produkt)
- Nicht ausreichende Qualität (z. B. instabile Software)
- Nicht erfüllte Anforderungen (z. B. falsche Spezifikationen in einer Dienstleistung)

Falschlieferung

Eine Falschlieferung bedeutet, dass ein anderes als das bestellte Produkt geliefert wurde. Ursachen können sein:

- Verwechslung des Produkts durch Fehler in der Logistik.
- Fehlkommunikation zwischen Auftraggeber und Auftragnehmer.
- Fehlerhafte Artikelnummern oder Bestellprozesse.

Minderlieferung

Eine Minderlieferung tritt auf, wenn die gelieferte Menge geringer ist als bestellt. Dies kann entstehen durch:

- Fehlberechnungen oder Verpackungsfehler.
- Lieferengpässe oder Produktionsprobleme.
- Fehlende Kommunikation über Teillieferungen.

7.15 Leistungserbringung

Die Leistungserbringung umfasst alle Tätigkeiten, die zur Erfüllung eines Auftrags oder Projekts notwendig sind. Dabei wird überprüft, ob die erbrachte Leistung den vereinbarten Anforderungen entspricht.

Soll-Ist-Vergleich

Der Soll-Ist-Vergleich dient dazu, die tatsächliche Leistung mit den geplanten Vorgaben zu vergleichen:

- **Soll-Werte:** Geplante Kosten, Zeitaufwand, Qualität und Ressourcen.
- **Ist-Werte:** Tatsächlich aufgewendete Mittel und erzielte Ergebnisse.
- **Abweichungen:** Differenzen zwischen Soll und Ist werden analysiert.

Info

Ein regelmäßiger Soll-Ist-Vergleich hilft, frühzeitig Probleme zu erkennen und gegenzusteuern.

Abweichungsanalyse

Wenn der Soll-Ist-Vergleich Abweichungen zeigt, erfolgt eine Abweichungsanalyse:

- **Identifikation der Ursachen:** Warum kam es zu Abweichungen? (z. B. unerwartete Kosten, Verzögerungen, Fehler in der Planung)
- **Bewertung der Auswirkungen:** Welche Konsequenzen haben die Abweichungen für das Gesamtprojekt?
- **Maßnahmen zur Korrektur:** Anpassung von Ressourcen, Budget oder Zeitplan.

Nachkalkulation

Die Nachkalkulation dient der Überprüfung der tatsächlichen Kosten nach Projektabschluss:

- Vergleich von geplanten und realen Kosten (Material, Personal, Zeitaufwand).
- Analyse von Kostentreibern und Einsparpotenzialen.
- Optimierung der Kalkulation für zukünftige Projekte.

Generierung von Nachfolgeaufträgen

Nach erfolgreicher Leistungserbringung können weitere Aufträge entstehen:

- Zufriedene Kunden als Basis für Folgeaufträge (z. B. Wartungsverträge, Erweiterungen).
- Analyse des Kundenbedarfs für zukünftige Dienstleistungen oder Produkte.
- Proaktive Kundenkommunikation zur langfristigen Zusammenarbeit.

8. Software-Entwicklung

8.1 Variablen, Datentypen und -strukturen

Variablen

Eine Variable ist ein Speicherplatz, der einen bestimmten Wert aufnehmen kann. Variablen haben einen Namen, einen Datentyp und einen Wert.

Datentypen

Ein Datentyp beschreibt die Art von Daten, die in einer Variablen gespeichert werden können. Es gibt verschiedene Datentypen, wie z.B.:

- **Ganzzahlen (int):** ganze Zahlen, z.B. 1, 2, 3
- **Fließkommazahlen (float):** Zahlen mit Dezimalstellen, z.B. 3.14, -0.5
- **Zeichen (char):** einzelne Zeichen, z.B. 'a', 'A', '!'
- **Boolesche Werte (bool):** Wahrheitswerte, z.B. true, false
- **Zeichenketten (string):** Folgen von Zeichen, z.B. "Hallo", "Welt"

Datenstrukturen

Eine Datenstruktur ist eine Sammlung von Daten, die in einer bestimmten Struktur organisiert sind. Es gibt verschiedene Datenstrukturen, wie z.B.:

- **Arrays:** eine Sammlung von Werten eines bestimmten Datentyps, die durch einen Index zugänglich sind.
- **Listen (List):** eine Sammlung von Werten, die in einer bestimmten Reihenfolge gespeichert sind.
- **Verknüpfte Listen (Linked List):** eine Liste, bei der jeder Eintrag auf den nächsten Eintrag verweist.
- **Stacks:** eine Datenstruktur, die es ermöglicht, Elemente hinzuzufügen und zu entfernen, wobei das letzte hinzugefügte Element zuerst entfernt wird.

- **Queues:** eine Datenstruktur, die es ermöglicht, Elemente hinzuzufügen und zu entfernen, wobei das erste hinzugefügte Element zuerst entfernt wird.

Komplexe Datenstrukturen

Es gibt auch komplexe Datenstrukturen, wie z.B.:

- **Structs:** eine Sammlung von Variablen, die zusammen eine Einheit bilden.
- **Klassen:** eine Sammlung von Variablen und Funktionen, die zusammen eine Einheit bilden.
- **Hash-Tabelle (Hash Table):** eine Datenstruktur, die es ermöglicht, Werte über einen Schlüssel zu speichern und abzurufen.

8.2 Kontrollstrukturen

Folgestruktur

Eine Folgestruktur ist die sequenzielle (Schritt für Schritt) Ausführung von Anweisungen in der Reihenfolge, in der sie geschrieben wurden.

Auswahlstruktur

- **Einseitige Auswahl:** If-Struktur (nur eine Bedingung und deren Ausführung)
- **Zweiseitige Auswahl:** If-Else-Struktur (zwei mögliche Ausführungspfade)
- **Mehrstufige Auswahl:** If-Elif-Else-Struktur (mehrere Bedingungen und Pfade)
- **Fallunterscheidung:** Switch/Case oder Match-Statement

Wiederholungsstruktur (Schleifen)

1. Geschlossene Schleifen

- Feste Anzahl von Durchläufen (zählergesteuert)
- Beispiel: For-Schleife, foreach-Schleife

2. Offene Schleifen

- **Kopfgesteuerte Schleifen:** Bedingung wird am Anfang geprüft (while)

- Fußgesteuerte Schleifen: Bedingung wird am Ende geprüft (do-while)

8.3 Prozeduren und Funktionen

Einfache Funktionen

Einfache Funktionen haben keine Übergabeparameter und keinen Rückgabewert. Sie führen nur Code aus.

Funktionen mit Rückgabewert

Funktionen mit Rückgabewert haben keine Übergabeparameter, geben aber einen Wert zurück. Beispiel: Getter-Methoden oder Statusabfragen.

Funktionen mit Übergabeparameter

Funktionen mit Übergabeparameter akzeptieren einen oder mehrere Parameter, geben aber keine Werte zurück. Parameter können übergeben werden als:

- Call-By-Value: Übergabe einer Wertkopie (Original bleibt unverändert)
- Call-By-Reference: Übergabe einer Referenz/Adresse (Original kann verändert werden)

Funktionen mit Rückgabewert und Übergabeparameter

Diese Funktionen kombinieren beides: Sie akzeptieren Parameter und geben einen Wert zurück. Dies ist die flexibelste Form von Funktionen.

8.4 Objektorientierung

Objekt / Instanz

Eine Instanz ist ein konkretes Exemplar einer Klasse. Beispiel: "meinAuto" ist eine Instanz der Klasse "Auto" mit individuellen Attributwerten wie Farbe="rot".

Klasse

Eine Klasse ist ein Bauplan für Objekte mit gemeinsamer Struktur. Sie definiert, welche Attribute und Methoden alle Objekte dieser Klasse besitzen.

Methoden

Methoden sind Funktionen einer Klasse, die das Verhalten der Objekte definieren. Sie können Attribute verarbeiten und mit anderen Objekten interagieren.

Attribut

Attribute sind die Eigenschaften eines Objekts. Sie speichern die spezifischen Werte (Attributwerte), die den Zustand des Objekts beschreiben.

Kapselung

Kapselung bedeutet, dass Daten (Attribute) und Methoden in einer Klasse zusammengefasst und nach außen verborgen werden. Zugriff erfolgt nur über definierte Schnittstellen.

Polymorphie

Polymorphie ermöglicht es, dass eine Methode in verschiedenen Klassen unterschiedlich implementiert werden kann, aber den gleichen Namen behält.

Vererbung

Vererbung erlaubt es, Eigenschaften und Methoden einer existierenden Klasse (Elternklasse/Basisklasse) an neue Klassen (Kindklassen/Abgeleitete Klasse) weiterzugeben.

Public, Private und Protected

- **Public:** Zugriff von überall möglich
- **Private:** Zugriff nur innerhalb der eigenen Klasse
- **Protected:** Zugriff in eigener Klasse und Kindklassen

Assoziation, Aggregation und Komposition

- **Assoziation:** Lose Beziehung zwischen Klassen

- **Aggregation:** "hat ein"-Beziehung, Teile können unabhängig existieren
- **Komposition:** Starke "hat ein"-Beziehung, Teile können nicht unabhängig existieren

8.5 Bibliotheken und Frameworks

Bibliotheken

Eine Bibliothek ist eine Sammlung von vordefinierten Funktionen und Klassen, die von einem Programm verwendet werden können. Bibliotheken erleichtern die Entwicklung von Programmen, indem sie wiederkehrende Aufgaben übernehmen und auslagern.

Frameworks

Ein Framework ist eine umfassende Sammlung von Bibliotheken, Tools und Konventionen, die die Entwicklung von Programmen erleichtern. Frameworks bieten eine Struktur für die Entwicklung von Anwendungen und erleichtern die Wiederverwendung von Code.

Vorteile

- Erleichtern die Entwicklung von Programmen
- Reduzieren die Entwicklungszeit
- Erhöhen die Wiederverwendbarkeit von Code
- Bieten eine umfassende Funktionalität

8.6 Skriptsprachen

Skriptsprachen sind interpretierte Sprachen, welche zudem meist auf gewisse Sprachelemente wie z.B. die Deklaration von Variablen. Sie werden hauptsächlich für kleine Programme und Algorithmen benutzt.

Ein Beispiel hierfür ist Shellscript.

Shellscript

Ein Shellskript ist ein Skript, das vom Shell-Interpreter ausgeführt wird. In der ersten Zeile eines Skriptes wird mit `#!/bin/sh` oder ähnlichem deklariert, welcher Interpreter das Skript ausführen soll. Die Dateierweiterung für Shellskripte ist typischerweise `.sh`.

Ein einfaches Beispiel für ein Shellskript könnte wie folgt aussehen:

```
#!/bin/sh
echo "Hallo Welt!"
```

Dieses Skript gibt einfach den Text "Hallo Welt!" auf der Konsole aus.

Ein weiteres Beispiel könnte ein Skript sein, das eine Liste von Dateien in einem Verzeichnis auflistet:

```
#!/bin/sh
ls -l
```

Dieses Skript führt den Befehl `ls -l` aus, der eine detaillierte Liste der Dateien im aktuellen Verzeichnis anzeigt.

8.7 Pseudocode

Verzweigungen

- IF-Anweisung:

```
IF Bedingung THEN
    Anweisung(en)
END IF
```

- IF-ELSE:

```
IF Bedingung THEN
    Anweisung(en)
ELSE
    Anweisung(en)
END IF
```

- IF-ELSE IF-ELSE:

```
IF Bedingung1 THEN
    Anweisung(en)
ELSE IF Bedingung2 THEN
    Anweisung(en)
ELSE
    Anweisung(en)
END IF
```

- **SWITCH/CASE:**

```
SWITCH Variable
    CASE Wert1:
        Anweisung(en)
        BREAK
    CASE Wert2:
        Anweisung(en)
        BREAK
    DEFAULT:
        Anweisung(en)
END SWITCH
```

Schleifen

- **WHILE-Schleife:**

```
WHILE Bedingung DO
    Anweisung(en)
END WHILE
```

- **REPEAT-UNTIL-Schleife:**

```
REPEAT
    Anweisung(en)
UNTIL Bedingung
```

- **FOR-Schleife:**

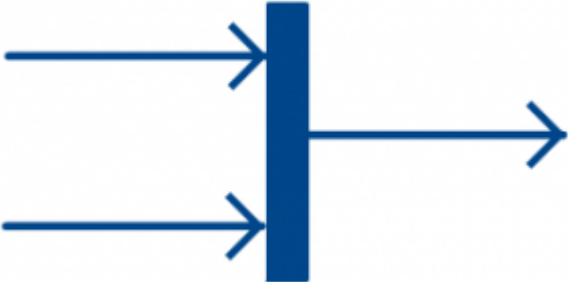
```
FOR Variable = Startwert TO Endwert DO
    Anweisung(en)
END FOR
```

8.8 UML

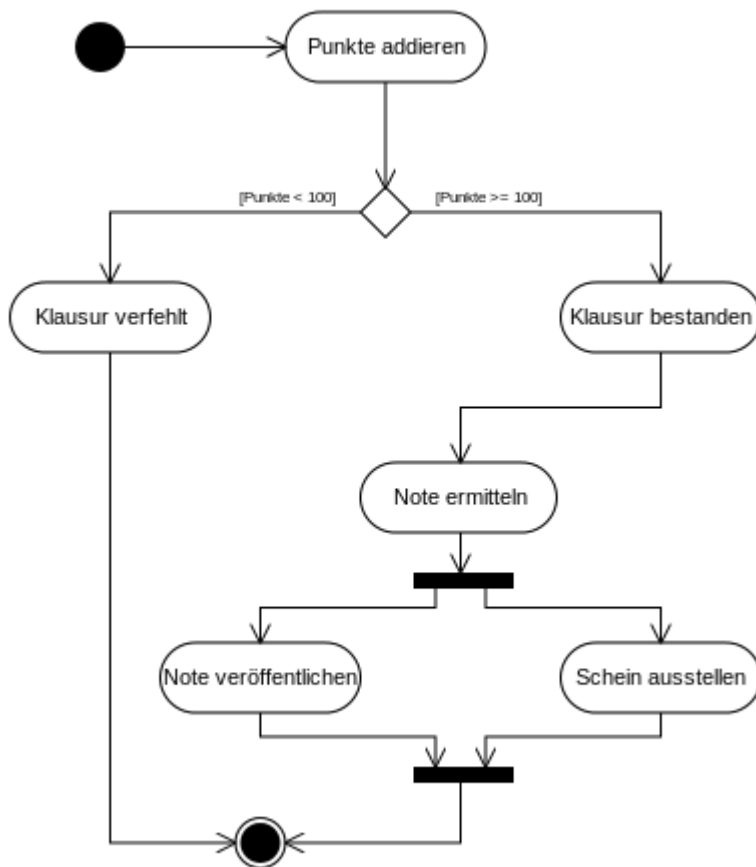
Aktivitätsdiagramm

Ein Aktivitätsdiagramm ist eine Art von UML-Diagramm, das den Ablauf von Aktivitäten und Aktionen innerhalb eines Systems veranschaulicht. Es wird verwendet, um den Workflow eines Systems zu modellieren und die Schritte zu identifizieren, die in einem Prozess involviert sind.

Symbol	Name
	Start/Anfangsknoten
	Aktivität/Aktionsstatus
	Aktion
	Kontrollfluss/Kante
	Objektfluss/Steuerkante
	Aktivität Endknoten
	Fluss-Endknoten
	Entscheidungs-Knotenpunkt

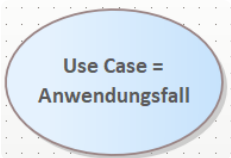
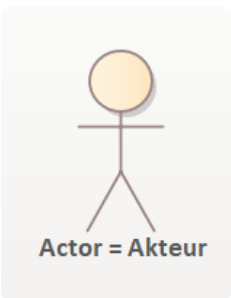
Symbol	Name
	Knoten verschmelzen
	Gabel
	Zusammenführen
	Senden von Signalen
	Signal-Empfang
	Anmerkung/Kommentar

Beispiel:



Anwendungsfalldiagramm

Ein Anwendungsfalldiagramm ist ein UML-Diagramm, das die Interaktionen zwischen einem System und seinen Benutzern zeigt. Es wird verwendet, um die funktionalen Anforderungen eines Systems zu identifizieren und die Beziehungen zwischen Akteuren und Anwendungsfällen zu modellieren.

Symbol	Bedeutung
	Anwendungsfall - Eine Funktion oder ein Prozess, der vom System bereitgestellt wird.
	Akteur - Eine Person oder ein System, das einen Anwendungsfall auslöst.